Second Sig: build spec v2

Oct 22, 2026 · @Rafi

Supersedes the 22 October spec. Three scenarios, one web UI, everything on a local mainnet fork running the real Safe contracts.

What we're building

Second Sig is a signature the humans can't give: a judge rehearses the exact bytes of a Safe proposal on a fork of Ethereum, compares what they do with what the proposer said, and signs an EIP-712 stamp only on a match. A guard on the Safe refuses any transaction without a stamp for its exact hash. Owners still sign as normal; their signatures alone no longer execute anything.

Everything on the fork is real. The Safe is a real Safe v1.4.1 proxy created through the real `SafeProxyFactory` on a pinned-block fork of mainnet. The guard is installed with a real `execTransaction(setGuard)`. Every execution goes through the real `Safe.sol`, which calls our guard. The contracts the scenarios touch sit at their mainnet addresses: Aave v3, USDC, MultiSendCallOnly, and the contract the Bybit attackers used. The only contract we write is `SecondSigGuard.sol`. That is what separates this from "an LLM reads the code": the chain enforces the verdict.

What we claim. The guard makes the check mandatory. A hard floor in code rejects the Bybit class before any model runs. Gemini 4 Argon, reading the real contract source, lets honest but complex work through and asks about the rest. **What we don't claim:** that it catches everything, or that it replaces signers.

Shape of the build. Two people, 36 working hours, Encode London, the Gemini 4 Argon challenge plus Agents That Ship. One web UI on a laptop talking to a local Anvil fork, three scenarios, one three-minute demo. Judges are likely AI engineers, not EVM auditors, so the UI explains as it goes.

The three scenarios

One per verdict. All three run against the same 2-of-3 Safe on the fork, with two owners' signatures already attached, so the point lands every time: the humans signed, and the chain still did what the judge said.

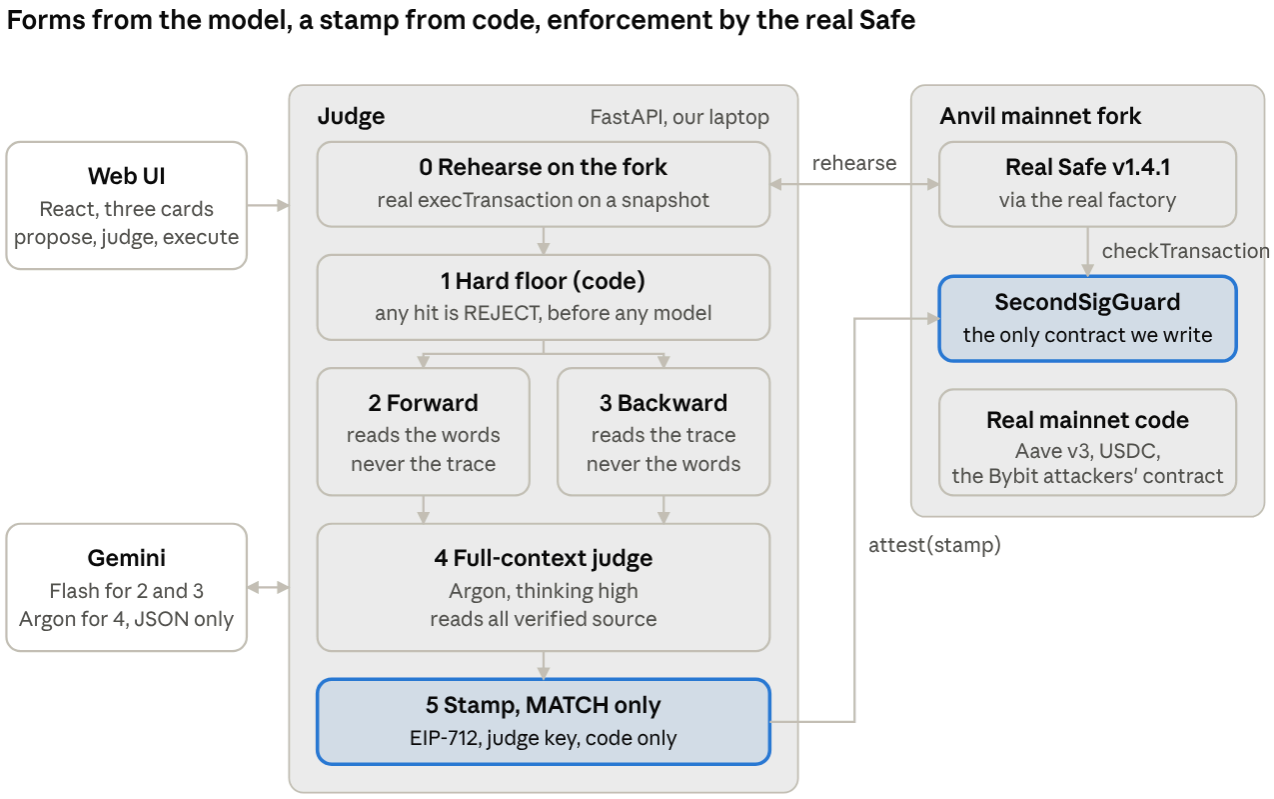
#	What the signers are told	What the bytes do	Verdict	What it shows
1	"Transfer 500 ETH to the warm wallet"	`delegatecall` to the Bybit attackers' contract with `transfer(0xbDd0…9516, 0)`, which writes the Safe's slot 0: its master copy	REJECT. Floor hits before any model; Argon agrees. Guard reverts with two valid signatures	The $1.4B attack shape, against a real Safe
2	"Deposit 10,000 USDC into Aave v3"	MultiSendCallOnly batch: `USDC.approve(Pool, 10,000e6)` then `Pool.supply(USDC, 10,000e6, safe, 0)`	MATCH. Stamp written, executes, Safe holds aUSDC	Complex honest DeFi passes; Argon has read the real Aave source
3	"Pay our auditor 100 USDC"	Batch: `USDC.transfer(auditor, 100e6)` then `USDC.approve(auditor, type(uint256).max)`	ASK, one question. No stamp, execution reverts	True but incomplete; no rule catches it without false alarms

Fixtures. Each scenario is a JSON file: the Safe transaction struct, the description, and an address book (warm wallet, auditor, Aave Pool, USDC). The chain lead builds the calldata; the addresses below are from the NCC analysis and the earlier reference sheet, and get re-checked against Etherscan and `safe-deployments` on Friday.

Thing	Address or value
Bybit attackers' contract (the delegatecall target)	`0x96221423681A6d52E184D440a8eFCEbB105C7242`
Address it writes into slot 0	`0xbDd077f651EBe7f7b3cE16fe5F2b025BE2969516`
Bybit cold wallet, for the judge-only replay (v1.1.1, no guard support)	`0x1Db92e2EeBC8E0c075a02BeA49a2935BcD2dFCF4`, attack at block 21,895,238
Safe v1.4.1 L1 singleton	`0x41675C099F32341bf84BFc5382aF534df5C7461a`
SafeProxyFactory, CompatibilityFallbackHandler, MultiSendCallOnly (v1.4.1)	From `safe-deployments`, Friday

Thing	Address or value
Aave v3 Pool, USDC, aUSDC	`0x87870Bca…fA4E2`, `0xA0b86991…6eB48`, `0x98C23E9d…6F5c`: confirm full addresses Friday
Fork block	Pinned at setup time, written to `.env`. Any block after 21,895,238 has every contract above

Architecture



Second Sig architecture · one laptop, one fork, one new contract

The UI holds the owners' test keys and drives the judge over HTTP; the judge drives the fork; only step 5, which is code, can write a stamp, and the real Safe's `execTransaction` asks the guard for it on every execution.

The judge pipeline

Five steps in a fixed order. The first two are code; the last three are model calls that return pydantic forms through `response_schema` and nothing else. Any failure anywhere (invalid form, timeout, model error) ends in ASK, never MATCH.

0. Rehearse (code). `evm_snapshot`; zero the guard slot with `anvil_setStorageAt` so the rehearsal isn't blocked by our own guard; run the real `execTransaction` with the owners' signatures; `debug_traceTransaction` with `callTracer` for the call tree and `prestateTracer` in diff mode for storage, falling back to SSTOREs from the struct log if the installed anvil lacks it; decode calls and events with ABIs from Etherscan; label the slots we know; `evm_revert`. Output: `EvidenceBundle`.

1. Hard floor (code). Rules with no honest counterexample. Any hit is REJECT before a model runs, whatever a model later says.

1. `delegatecall` to anything other than the canonical MultiSendCallOnly
2. A write to the Safe's own slot 0 (singleton), guard slot, fallback-handler slot or modules mapping
3. A call or delegatecall into a contract with no verified source
4. `gasPrice != 0` with a gas token or refund receiver set (a refund drain)

Bybit trips 1, 2 and 3. Unlimited approval is deliberately not here: it's sometimes intended, so it's a question for the model, which is scenario 3. Cheap additions if there's time: `approve` or `setApprovalForAll` to an EOA; a recipient that matches an address-book entry at both ends but not in the middle.

2. Forward pass (`gemini-3.8-flash`, blind to the trace). Input: the description, spotlighted, plus the address book. Output: `ExpectedEffects`, a list of effects the proposer's words imply (asset, from, to, amount, kind) and a `nothing_else` flag meaning the words don't license anything beyond the list.

3. Backward pass (`gemini-3.8-flash`, blind to the description). Input: decoded call tree, labelled storage diff, balance changes. Output: `Narration`, one plain-English sentence per effect with a severity (`info`, `notable`, `critical`) and an index into the bundle. It never sees the proposer's text, so the text can't steer it. Runs in parallel with step 2.

4. Full-context judge (`gemini-4-argon`, `thinking_level: high`). Input: expected effects, narration, the raw call tree and diff, and the verified source of every contract touched, proxies resolved to their implementations. Output: `Verdict`: `MATCH | ASK | REJECT`, `reasoning`, `question` (ASK only), `discrepancies`. This is where the model earns its place: Aave's dozens of storage writes are a normal deposit because the source says so; an unmentioned unlimited approval is a question; slot 0 of a Safe changing is a hijack.

Decision. REJECT if the floor hit; otherwise Argon's verdict. The stamp is signed on MATCH only, by code, with a key the model never sees.

Gemini notes. Interactions API via `google-genai`; `response_schema` from pydantic on every call; thinking summaries on for the judge call if the flag is available on Argon (workshop question), otherwise we stream the `reasoning` field as it arrives; temperature is ignored on Argon, so every interaction id is logged and `--replay` rebuilds any verdict from `logs/` without a model call. Measure the Aave source with `count_tokens` on Saturday; if the full set is over about 600k tokens, send only the files containing functions that appear in the trace.

The real Safe on the fork, the guard, the stamp

The demo Safe is created the way any Safe is, through the real factory, and the guard is installed the way any guard is, with a Safe transaction. Nothing is mocked.

Setup script, run once (`contracts/script/SetupSafe.s.sol` or a Python equivalent).

1. `anvil --fork-url $MAINNET_RPC --fork-block-number $FORK_BLOCK`
2. `SafeProxyFactory.createProxyWithNonce(singleton_1_4_1, setup(owners=[o1,o2,o3], threshold=2, fallbackHandler=CompatibilityFallbackHandler, ...), salt)`
3. Fund it: 1,000 ETH with `anvil_setBalance`; 50,000 USDC by impersonating a large holder and transferring
4. Deploy `SecondSigGuard(safe, judgeAddress, JUDGE_VERSION)`
5. `execTransaction(to=safe, data=setGuard(guard))`, signed by o1 and o2. From here on, nothing executes without a stamp
6. `evm_snapshot` for the base state the UI's Reset button returns to. Run all three scenarios once online: Foundry's on-disk RPC cache for a pinned block then serves the demo with no network

`SecondSigGuard.sol`, about 80 lines. Inherits `BaseGuard` from the v1.4.1 tag so `setGuard`'s ERC-165 check passes.

- `attest(Stamp s, bytes sig)`: recover the EIP-712 signer; require it is the judge and `s.judgeVersion == JUDGE_VERSION`; store `stamps[s.safeTxHash] = s.forkBlock`; emit `Attested`. Anyone may submit it.
- `checkTransaction(to, value, data, operation, safeTxGas, baseGas, gasPrice, gasToken, refundReceiver, signatures, msgSender)`: require `msg.sender == SAFE`; recompute `hash = SAFE.getTransactionHash(..., SAFE.nonce() - 1)`, because the Safe has already incremented its nonce when it calls the guard; if `overrides[hash]`, pass; else require `stamps[hash] != 0` and `block.number - stamps[hash] <= MAX_STAMP_AGE`, else `revert SecondSig__NoStamp(hash)`.

- `checkAfterExecution(bytes32, bool)`: empty. Optional three lines: revert if the Safe's slot 0 changed.
- `approveOverride(bytes32 hash)`: each current owner calls it directly (`SAFE.isOwner(msg.sender)`); when every owner has, `overrides[hash] = true`. No timelock in this build. It exists so a dead judge can never brick the Safe, and it is the Q&A answer, not a demo beat.
- Custom errors `SecondSig__...`, events for every attest and override approval.

The stamp (EIP-712). Domain `{name: "SecondSig", version: "1", chainId, verifyingContract: guard}`. Type `Stamp(bytes32 safeTxHash, uint64 forkBlock, bytes32 judgeVersion)`. Python signs with `eth_account.sign_typed_data`; the Foundry tests sign with `vm.sign`. Binding `forkBlock` means a stamp for a rehearsal that happened on stale state expires (`MAX_STAMP_AGE = 100` in the demo).

Foundry tests, green before the guard is set on the fork.

- [ ] Stamp for hash A does not pass hash B; a stamp cannot be replayed at a later nonce
- [ ] Wrong signer, wrong `judgeVersion`, or stale `forkBlock` reverts
- [ ] `setGuard(address(0))` without a stamp reverts: an attacker's first move would be removing the guard
- [ ] All three owners call `approveOverride`; the transaction then executes with no stamp
- [ ] The Bybit calldata with two valid signatures reverts at the guard; the Aave batch with a stamp executes

Why v1.4.1 and not the Bybit Safe itself. Bybit's Safe was v1.1.1, which predates guards. The judge can still be run on the real attack transaction for the pitch; the guard demo replays the same shape against our own Safe. The attackers' contract is still on mainnet, so the fork delegatecalls the real thing.

The three interfaces between us

Agreed Friday night as pydantic models in `judge/schemas.py`, then frozen. Each of us builds against a hand-written example of the other's output until the real one exists.

```
class EvidenceBundle(BaseModel):          # chain lead produces, Rafi
consumes
    safe_tx: SafeTx                        # to, value, data, operation, gas
fields, nonce
    safe_tx_hash: str
    fork_block: int
    call_tree: list[Call]                  # depth, from, to, selector,
decoded name + args, value, success
    storage_diff: list[SlotWrite]          # contract, slot, label | None,
```

```
before, after
    balance_diff: list[BalanceChange]      # address, asset, delta
    events: list[Event]
    sources: dict[str, Source]             # address -> name, verified
source, abi, implementation_of
    floor_hits: list[FloorHit]             # rule id, evidence index, one-
line reason

class Verdict(BaseModel):                  # Rafi produces, UI and stamp
consume
    verdict: Literal["MATCH", "ASK", "REJECT"]
    reasoning: str
    question: str | None
    discrepancies: list[str]
    floor_hits: list[FloorHit]

class Stamp(BaseModel):                    # Rafi signs, guard verifies
    safe_tx_hash: str
    fork_block: int
    judge_version: str                     # bytes32 hex, equals the guard's
JUDGE_VERSION
```

The fourth boundary is the HTTP API between the UI and the judge; that lives in the UI spec.

Roles

The chain lead owns everything that touches bytes, state or Solidity; Rafi owns everything that touches text, models or pixels. Neither waits on the other mid-phase: the schemas above are the only handshake.

Area	Chain lead	Rafi
Fork and Safe setup	Pinned fork, real Safe v1.4.1 via the factory, funding, `setGuard`, base snapshot, warm RPC cache	
Rehearsal and evidence	Snapshot, guard-slot zeroing, `execTransaction` with signatures, both traces, decoding, slot labels, balance diff	
Hard floor	The rule list and its tests: he knows what exploits look like	Wires `floor_hits` into the decision and the UI

Area	Chain lead	Rafi
Source fetch	Etherscan `getsourcecode`, proxy resolution, ABI cache	Token budget and trimming for the judge call
Guard and stamp	`SecondSigGuard.sol`, Foundry tests, `attest` and `approveOverride`, stamp verification	`Stamp` signing in Python, `JUDGE_VERSION`
Fixtures	Calldata for all three scenarios, the address book	Descriptions and context banner copy
Execution	`/api/execute`: real `execTransaction`, decoded custom-error on revert, receipt on success; `/api/reset`	
Model stages		Forward, backward, judge prompts and schemas; Gemini adapter; streaming; `--replay`
API and UI	Reviews the plain-English slot translations for accuracy	FastAPI, SSE, React app, the three cards
Pitch	Guard and chain questions in Q&A; records the on-chain beats	Script, video cut, model questions in Q&A

Working agreements. `main` always runs all three scenarios; half-built work stays on a branch. Ten-minute stand-up at each gate. Both of us use Claude Code with the shared `CLAUDE.md`, which carries the invariants, so neither agent quietly breaks the other half. Whoever owns a part demos it and answers for it.

36-hour plan

The gate that matters is Saturday midday: if the Bybit shape isn't rejected by the floor and reverted by the guard on the fork by then, everything else waits until it is.

Gate	When	Chain lead has	Rafi has	If it slips
G0	Friday night	Pinned fork; real Safe deployed, funded, guard set; guard compiles with tests stubbed	One Flash and one Argon call through the adapter with `response_schema`, logged; FastAPI skeleton with a fake pipeline; Vite app with three empty cards	Stay until the Safe exists on the fork; the rest can move
G1	Saturday midday	Bybit fixture rehearsed and traced; decoded tree and labelled diff; floor fires; Foundry tests green, including the Bybit revert	Forward and backward run on the Bybit bundle; cards 1 and 2 show real output over SSE	Drop scenario 3 and the reasoning stream; both on G1
G2	Saturday night	Aave fixture; source fetch with proxies resolved; USDC and Aave slot labels; `/api/execute` returns a receipt or a decoded revert	Argon judge with full context; token budget measured; reasoning streams in card 3; stamp signed; MATCH, attest, execute end to end in the UI	Send ABI and function names to Argon instead of full source
G3	Sunday morning	Auditor fixture; `/api/reset`; RPC cache warm; all three scenarios run offline	ASK path in the UI; context banners; demo script; clips recorded	Scenario 3 becomes a recorded clip or is dropped
Freeze	3 hours before submission	Rehearse three times, cut the video, write the README. No new features		

Cut list, first to go at the top.

1. Live reasoning stream (show the final reasoning text instead)
2. Scenario 3
3. Full source to Argon (ABI, function names and storage layout only)
4. UI polish beyond the three cards and the banner

5. Voice-over (burned-in captions only)

Never cut. The real Safe on the fork with the guard set through `execTransaction`. The Bybit replay: REJECT, then the on-chain revert with two valid signatures. The Aave deposit: MATCH, stamp, execution. The three cards.